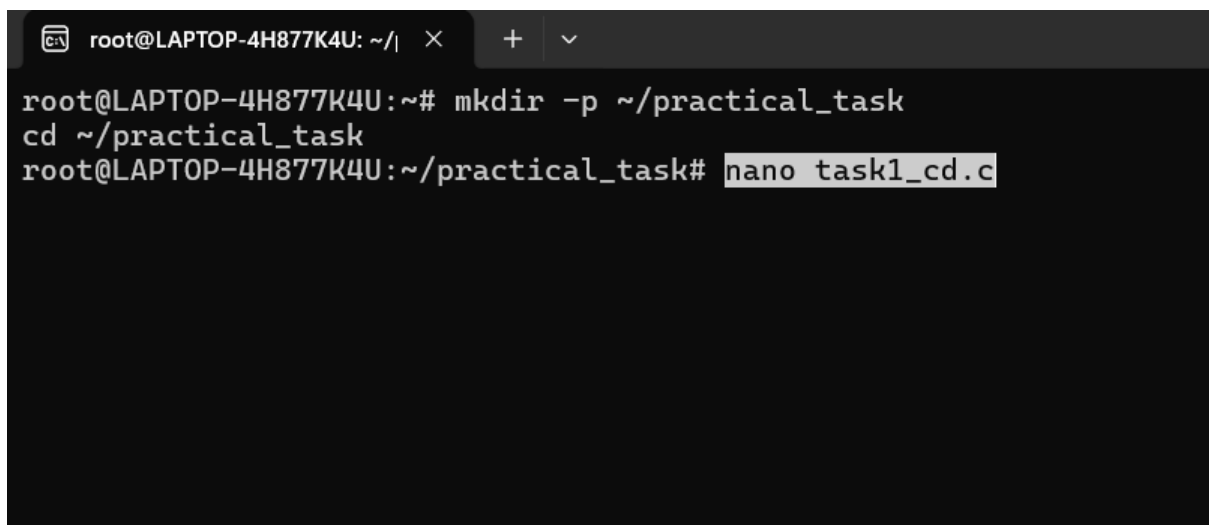


2520090008_ Sathish kumar

Skill – 5:

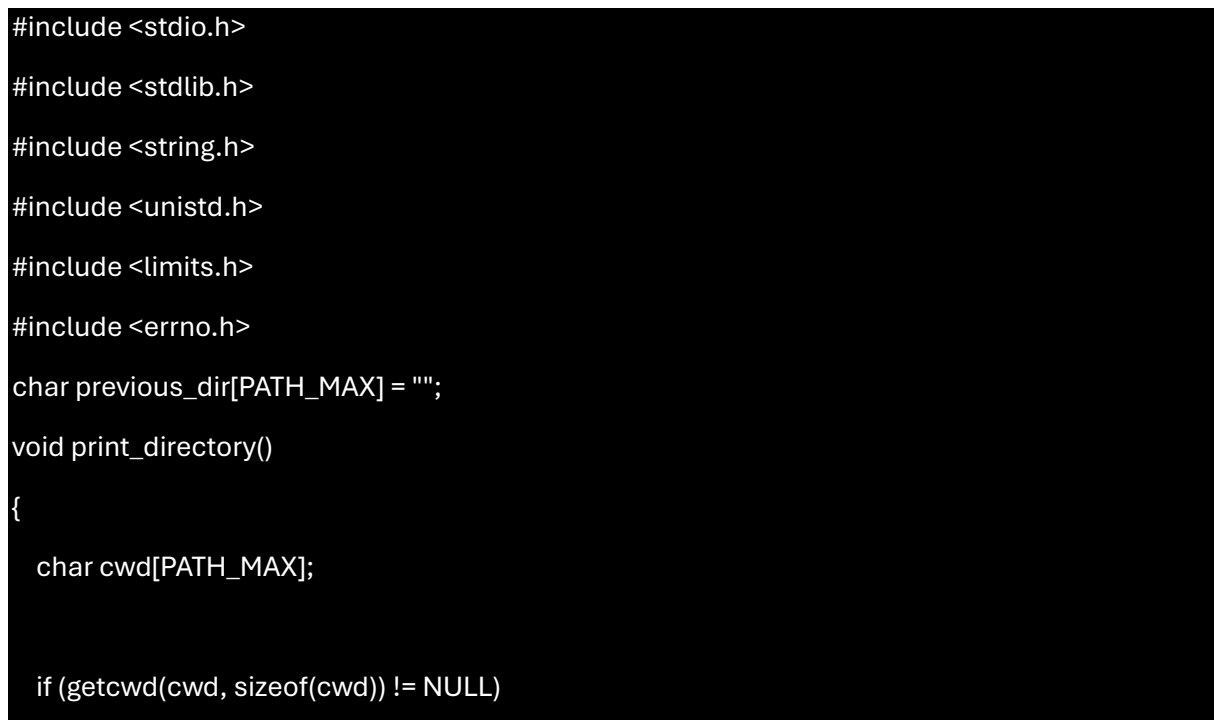
To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

Creating File:

A terminal window with a dark background. The title bar shows 'root@LAPTOP-4H877K4U: ~/l' with a close button. The terminal content shows the following commands and output:

```
root@LAPTOP-4H877K4U:~# mkdir -p ~/practical_task
cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano task1_cd.c
```

Code:

A code editor window with a dark background. It contains the following C code snippet:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>

char previous_dir[PATH_MAX] = "";

void print_directory()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
```

```

    printf("Current Directory: %s\n", cwd);
else
    perror("getcwd");
}

void change_directory(char *path)
{
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("getcwd");
        return;
    }
    if (chdir(path) == 0)
    {
        strcpy(previous_dir, cwd);
        printf("Directory changed successfully.\n");
        print_directory();
    }
    else
    {
        printf("Error: Cannot change to '%s'\n", path);
        perror("Reason");
    }
}

void go_previous()
{
    if (strlen(previous_dir) == 0)
    {
        printf("No previous directory available.\n");
        return;
    }

```

```

char temp[PATH_MAX];

strcpy(temp, previous_dir);

char cwd[PATH_MAX];

if (getcwd(cwd, sizeof(cwd)) == NULL)
{
    perror("getcwd");
    return;
}

if (chdir(temp) == 0)
{
    strcpy(previous_dir, cwd);
    printf("Moved to previous directory.\n");
    print_directory();
}
else
{
    perror("chdir");
}
}

int main()
{
    char command[PATH_MAX];

    printf("Directory Navigation Program\n");
    printf("Commands: pwd, cd <path>, prev, exit\n");

    while (1)
    {
        printf("nav> ");
        fflush(stdout);
    }
}

```

```
if (fgets(command, sizeof(command), stdin) == NULL)
```

```
    break;
```

```
command[strcspn(command, "\n")] = '\0';
```

```
if (strcmp(command, "exit") == 0)
```

```
{
```

```
    break;
```

```
}
```

```
else if (strcmp(command, "pwd") == 0)
```

```
{
```

```
    print_directory();
```

```
}
```

```
else if (strcmp(command, "prev") == 0)
```

```
{
```

```
    go_previous();
```

```
}
```

```
else if (strncmp(command, "cd ", 3) == 0)
```

```
{
```

```
    char *path = command + 3;
```

```
    while (*path == ' ')
```

```
        path++;
```

```
    if (strlen(path) == 0)
```

```
        printf("Error: Path not specified.\n");
```

```
    else
```

```
        change_directory(path);
```

```
}
```

```
else if (strcmp(command, "cd") == 0)
```

```
{
```

```
    const char *home = getenv("HOME");
```

```

    if (home != NULL)
        change_directory((char *)home);
    else
        printf("HOME variable not found.\n");
}
else
{
    printf("Invalid navigation command.\n");
}
}
return 0;
}

```

Output:

```

root@LAPTOP-4H877K4U:~/practical_task# nano task1_cd.c
root@LAPTOP-4H877K4U:~/practical_task# gcc task1_cd.c -o task1_cd
root@LAPTOP-4H877K4U:~/practical_task# ./task1_cd
Directory Navigation Program
Commands: pwd, cd <path>, prev, exit
nav> pwd
Current Directory: /root/practical_task
nav> cd/tmp
Invalid navigation command.
nav> pwd
Current Directory: /root/practical_task
nav> cd /etc
Directory changed successfully.
Current Directory: /etc
nav> pwd
Current Directory: /etc
nav> prev
Moved to previous directory.
Current Directory: /root/practical_task
nav> pwd
Current Directory: /root/practical_task
nav> exit

```

Task-2:

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Creating file:

```
root@LAPTOP-4H877K4U:~/practical_task# cd ~/practical_task
nano task2_builtin.c
```

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define INPUT_SIZE 256

typedef void (*command_function)(char *);
char previous_directory[PATH_MAX] = "";
/* =====
Built-in: pwd
===== */
void builtin_pwd(char *args)
{
    char current_directory[PATH_MAX];
    if (getcwd(current_directory, sizeof(current_directory)) != NULL)
    {
        printf("%s\n", current_directory);
    }
    else
    {
        perror("pwd");
    }
}
```

```

    }
}

/* =====

Built-in: cd

===== */

void builtin_cd(char *args)
{
    char current_directory[PATH_MAX];

    /* Get current directory */

    if (getcwd(current_directory, sizeof(current_directory)) == NULL)
    {
        perror("getcwd");

        return;
    }

    /* Remove leading spaces */

    while (*args == ' ')
    {
        args++;
    }

    /* cd without argument -> HOME */

    if (*args == '\0')
    {
        args = getenv("HOME");

        if (args == NULL)
        {
            printf("HOME directory not found\n");

            return;
        }
    }

    /* cd ~ -> HOME */

    if (strcmp(args, "~") == 0)

```

```

{
    args = getenv("HOME");
    if (args == NULL)
    {
        printf("HOME directory not found\n");
        return;
    }
}

/* cd - -> previous directory */
if (strcmp(args, "-") == 0)
{
    if (strlen(previous_directory) == 0)
    {
        printf("cd: previous directory not set\n");
        return;
    }
    args = previous_directory;
}

/* Change directory */
if (chdir(args) == -1)
{
    perror("cd");
    return;
}

/* Save previous directory */
strcpy(previous_directory, current_directory);

/* Display new directory */
if (getcwd(current_directory, sizeof(current_directory)) != NULL)
{
    printf("Current directory: %s\n", current_directory);
}

```

```

}

/* =====

Built-in: echo

===== */

void builtin_echo(char *args)
{
    while (*args == ' ')
    {
        args++;
    }

    printf("%s\n", args);
}

/* =====

Built-in: help

===== */

void builtin_help(char *args)
{
    printf("\n");
    printf("===== Built-in Commands =====\n");
    printf("cd <directory> - Change directory\n");
    printf("cd      - Go to HOME directory\n");
    printf("cd ~    - Go to HOME directory\n");
    printf("cd -    - Go to previous directory\n");
    printf("pwd     - Display current directory\n");
    printf("echo <text> - Display text\n");
    printf("help    - Display available commands\n");
    printf("exit    - Exit the shell\n");
    printf("===== \n");
}

/* =====

```

Built-in: exit

===== */

```
void builtin_exit(char *args)
```

```
{
```

```
    printf("Exiting shell...\n");
```

```
    exit(0);
```

```
}
```

/* =====

Dispatch Table

===== */

```
typedef struct
```

```
{
```

```
    char *name;
```

```
    command_function function;
```

```
} Command;
```

```
Command command_table[] =
```

```
{
```

```
    {"cd", builtin_cd},
```

```
    {"pwd", builtin_pwd},
```

```
    {"echo", builtin_echo},
```

```
    {"help", builtin_help},
```

```
    {"exit", builtin_exit}
```

```
};
```

/* Calculate number of commands */

```
int command_count =
```

```
    sizeof(command_table) / sizeof(command_table[0]);
```

/* =====

Main Function

===== */

```
int main()
```

```
{
```

```

char input[INPUT_SIZE];

printf("=====\n");

printf("    BUILT-IN COMMAND SHELL\n");

printf("=====\n");

while (1)
{
    printf("\nmyShell> ");

    fflush(stdout);

    /* Read command */

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("\n");

        break;
    }

    /* Remove newline */

    input[strcspn(input, "\n")] = '\0';

    /* Ignore empty input */

    if (strlen(input) == 0)
    {
        continue;
    }

    /* Get command */

    char *command = strtok(input, " ");

    /* Get arguments */

    char *args = strtok(NULL, "");

    if (args == NULL)
    {
        args = "";
    }

    int found = 0;

    /* Search dispatch table */

```

```

    for (int i = 0; i < command_count; i++)
    {
        if (strcmp(command, command_table[i].name) == 0)
        {
            command_table[i].function(args);
            found = 1;
            break;
        }
    }

    /* Invalid command */
    if (!found)
    {
        printf("Invalid command: %s\n", command);
    }
}

return 0;
}

```

Output:

```

root@LAPTOP-4H877K4U:~/practical_task# nano task2_builtin.c
root@LAPTOP-4H877K4U:~/practical_task# gcc task2_builtin.c -o task2_builtin
root@LAPTOP-4H877K4U:~/practical_task# ./task2_builtin

=====

    BUILT-IN COMMAND SHELL

=====

myShell> pwd

/root/practical_task

myShell> cd /tmp

Current directory: /tmp

myShell> pwd

/tmp

```

```
myShell> cd -
```

```
Current directory: /root/practical_task
```

```
myShell> echo Hello Ubuntu
```

```
Hello Ubuntu
```

```
myShell> help
```

```
===== Built-in Commands =====
```

```
cd <directory> - Change directory
```

```
cd          - Go to HOME directory
```

```
cd ~        - Go to HOME directory
```

```
cd -        - Go to previous directory
```

```
pwd         - Display current directory
```

```
echo <text>  - Display text
```

```
help        - Display available commands
```

```
exit        - Exit the shell
```

```
=====
```